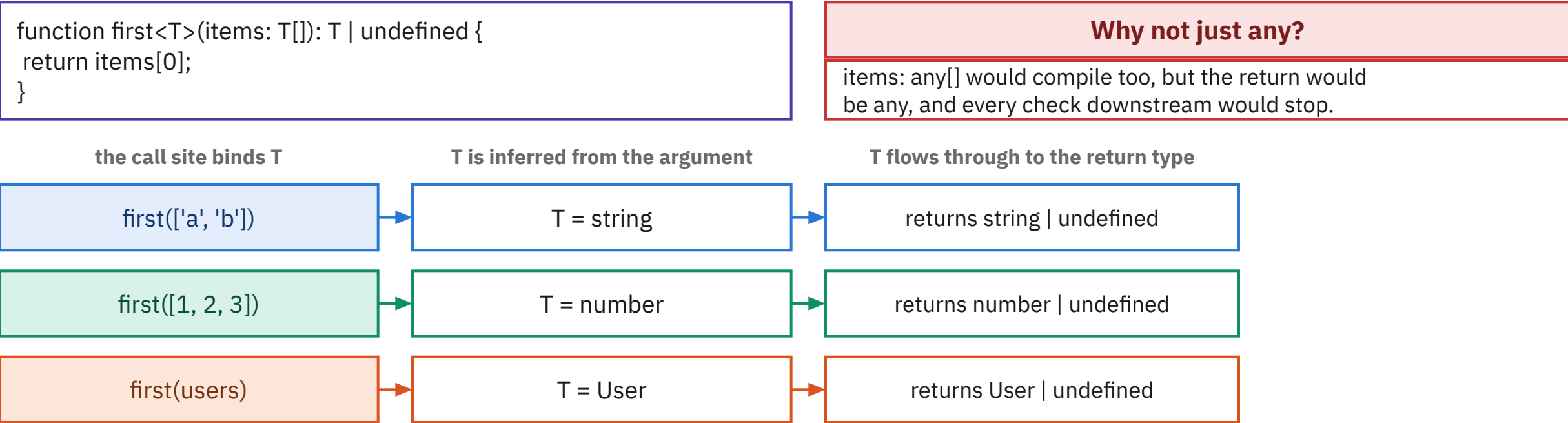


How a Type Parameter Flows

T is a placeholder filled in at the CALL. It carries the real type through to the return, so nothing is lost.



Constraints: extends is a GATE, not inheritance

```
function longest<T extends { length: number }>(a: T, b: T): T {  
  return a.length >= b.length ? a : b;  
}
```

Without the constraint, .length is an error: TypeScript knows nothing about T. With it, T still keeps the caller's exact type on the way out: pass two strings, get a string back.

Name a parameter for what it means (<Item>, <Key>) once you have more than one. Do not add a type parameter until something actually varies: a generic used in only one place is just a longer way to write the concrete type.